



Cerrojos



Un **cerrojo** es una **primitiva de control de la concurrencia** cuya misión es **garantizar la exclusión mutua**: solo un hilo puede ejecutar una sección crítica a la vez.

Explicación

Los cerrojos nos permiten evitar **condiciones de carrera**, protegiendo el acceso a recursos compartidos.

Un cerrojo es siempre **binario**, aunque su implementación interna sea más compleja. Tiene 2 estados:

- **Abierto** → disponible (unlock).
- **Cerrado** → ocupado por un hilo (lock/acquire).

Las operaciones:

- `.lock()` / `acquire()`
- `.unlock()` / `release()`

Son **atómicas**: si no lo fueran, dos hilos podrían entrar en la sección crítica a la vez simplemente porque la máquina entrelaza las instrucciones.

Cuando un hilo adquiere un cerrojo, **todos los demás hilos que intenten entrar se bloquean** hasta que el cerrojo se libere.

Veamos su implementación

Implementación

Java API Standart

Java, en su API estándar, **NO ofrece una clase "Lock" explícita.**

Si nos preguntan si tenemos cerrojos en esta versión, **NO TENEMOS**. Ya que no podemos usar una clase cerrojo o cosas así.

Sin embargo, *sí* ofrece un mecanismo de exclusión mutua incorporado en **todos los objetos del lenguaje**. Todos los objetos en Java tienen un cerrojo asociado que podemos manipular mediante la palabra reservada `synchronized`.

Cada objeto en Java tiene:

- un **cerrojo intrínseco** (monitor lock),
- un **wait-set** (cola de espera asociada a ese cerrojo).

```
public synchronized void incrementar() {  
    contador++;  
}
```

¿Qué hace exactamente `synchronized` ?

1. El hilo intenta adquirir el **lock intrínseco** del objeto.
2. Si otro hilo ya lo posee, este hilo se **bloquea** en la cola de exclusión mutua.
3. Cuando el hilo entra, ejecuta la sección crítica.
4. Al salir del bloque, **libera el cerrojo** automáticamente.

Cosas a tener en cuenta de `synchronized` :

- Es **reentrante** (para saber más, ver los apuntes sobre reentrancia):
si un hilo que ya tiene el cerrojo vuelve a pedir el mismo lock, entra sin bloquearse.
- El bloqueo se libera **siempre**, aunque haya excepciones.
- Si usas `wait()` , `notify()` , `notifyAll()` ,
debes hacerlo **dentro de un bloque synchronized sobre el mismo objeto** (ya que sino el compilador no sabe a qué cola de exclusión mutua estamos mandando las hebras).

Java API Alto Nivel

Java proporciona en el paquete `java.util.concurrent.locks` una serie de cerrojos más flexibles que `synchronized`.

`ReentrantLock`

Es el cerrojo más habitual.

Tiene semántica parecida a `synchronized`, pero con capacidades extra:

- Es **reentrante también** (para saber más, ver los apuntes sobre reentrancia).

Si un hilo que ya posee el cerrojo intenta tomarlo otra vez (por recursión o llamada interna), no se bloquea.

“El propietario eres tú; no compites contra ti mismo”.

- Permite:
 - **bloqueo explícito** (`lock()`)
 - **intentos no bloqueantes** (`tryLock()`)
 - **bloqueo con timeout** (`tryLock(timeout, unit)`)
 - **varias condiciones asociadas** mediante `newCondition()` (a diferencia de `synchronized`, que solo ofrece un único wait-set)

Ejemplo:

```
import java.util.concurrent.locks.*;

public class Contador {
    private int valor = 0;
    private final ReentrantLock lock = new ReentrantLock();

    public void incrementar() {
        lock.lock();      // toma el cerrojo
        try {
            valor++;
        } finally {
            lock.unlock(); // libera SIEMPRE el cerrojo
        }
    }
}
```

```
}  
}
```

C++

En C++, sí existe una API explícita de cerrojos desde C++11, en `<mutex>`.

Los ingredientes principales son:

- `std::mutex` → cerrojo **NO reentrante**

```
std::mutex m;  
  
void incrementar() {  
    m.lock();  
    contador++;  
    m.unlock();  
}
```

- `std::recursive_mutex` → cerrojo **reentrante**

```
#include <iostream>  
#include <mutex>  
  
std::recursive_mutex m;  
int contador = 0;  
  
void incrementar_recursoivo(int n) {  
    // El MISMO hilo bloquea el mismo mutex varias veces  
    m.lock(); // 1ª adquisición en la primera llamada  
  
    if (n <= 0) {  
        m.unlock();  
        return;  
    }  
  
    contador++; // sección crítica
```

```

// llamada recursiva: vuelve a hacer m.lock() sobre el mismo mutex
incrementar_recursivo(n - 1);

m.unlock();
}

int main() {
    incrementar_recursivo(5);
    std::cout << "contador = " << contador << std::endl; // debería mostrar 5
    return 0;
}

```

- `std::lock_guard` → adquiere y libera **automáticamente**

```

void incrementar() {
    std::lock_guard<std::mutex> guard(m);
    contador++;
} // ← se libera aquí automáticamente

```

- `std::unique_lock` → cerrojo flexible, permite desbloquear manualmente, pasar a condición, etc.

```

std::unique_lock<std::mutex> lock(m);
cv.wait(lock);

```